Contents

1 HMI Server 推荐方案 **1**
1.1 1. 结论 . . . 1
1.2 2. 为什么不建议继续停留在当前形态 . . . 1
1.3 3. 通信模式判断 . . . 1
1.4 4. 推荐目标架构 . . . 2
1.5 5. 方案对比 . . . 3
1.6 6. 最终推荐 . . . 3
1.6.1 6.1 Runtime / Northbound . . . 3
1.6.2 6.2 Device Integration / Southbound . . . 4
1.6.3 6.3 Data / Ops . . . 4
1.7 7. 协议与集成策略 . . . 4
1.8 8. 实施清单 . . . 4
1.8.1 第一阶段：必须做 . . . 4
1.8.2 第二阶段：建议尽快做 . . . 4
1.8.3 第三阶段：按规模再做 . . . 4
1.9 9. 本次整理产物 . . . 5
1.10 10. 参考 . . . 5

1 HMI Server 推荐方案

1.1 1. 结论

针对当前 HMI server 还处在最小 HTTP + JSON + polling 过渡态、后续又要接入 CNC / PLC / IPC 的情况，推荐采用一套“单站

- Runtime 继续保留 C++ 主体
- northbound 从手写 socket/HTTP 演进到 Drogon + HTTPS/REST + WebSocket
- 设备接入优先走 OPC UA
- 多品牌 legacy 协议先收口到工业网关，再由 runtime 消费统一语义
- 只有在标准协议不够时，才补厂商 SDK / 专有协议 适配
- 数据与运维先采用 PostgreSQL + OpenTelemetry + Prometheus + Grafana + Docker Compose + Traefik

这套方案不是“最重”的平台化设计，但对当前代码基线最友好，也最符合单站点边缘部署的真实节奏。

1.2 2. 为什么不建议继续停留在当前形态

当前实现作为原型和合同收敛阶段是够用的，但不适合直接作为正式运行时长期扩展，主要原因有四个：

1. HTTP + 高频轮询 会把高频状态、告警和异步命令结果都压到 request/response 模型里，实时性和成本都不理想。
2. hand-written northbound transport 后续会不断背负连接管理、超时、升级 WSS、鉴权、路由、中间件和观测成本。
3. southbound 如果直接把所有 PLC/CNC 协议都塞进主进程，后期协议种类一多，维护风险会远大于业务风险。
4. 设备原始地址空间和 HMI 语义模型没有分层时，前端和设备细节会被直接耦合。

1.3 3. 通信模式判断

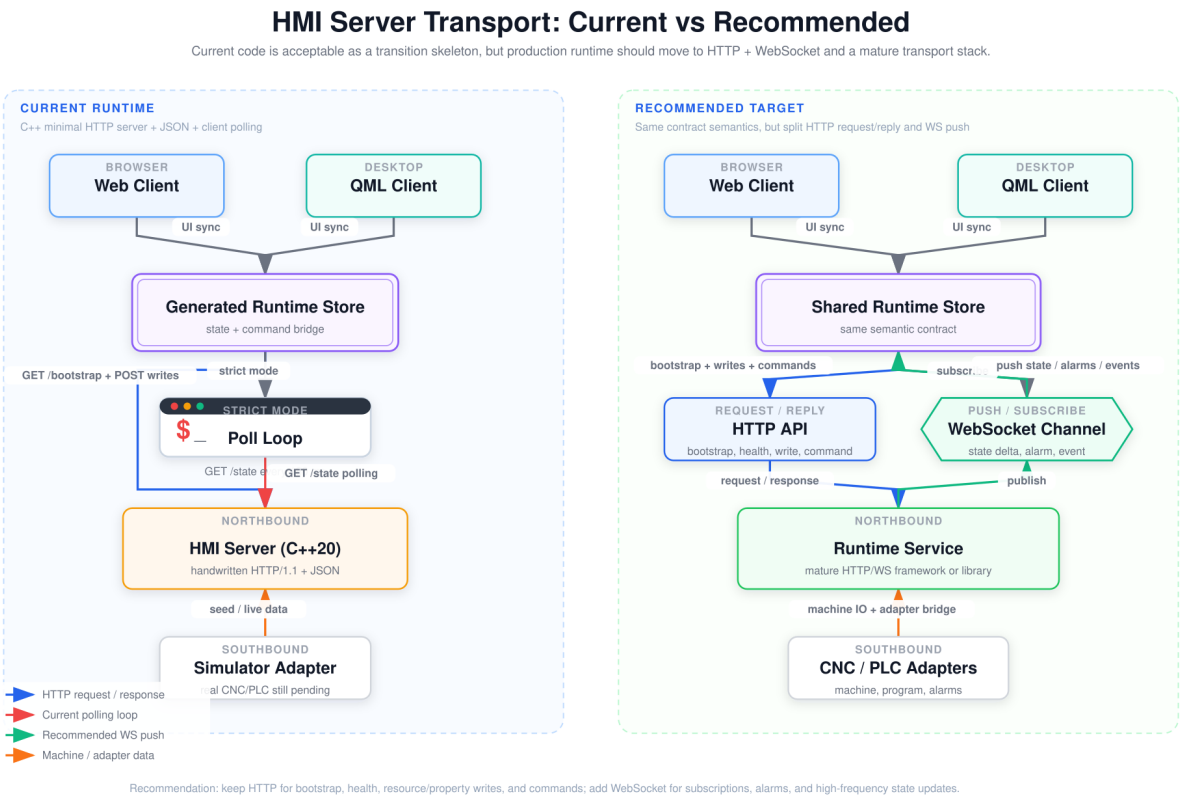
推荐保留 HTTP，但只让它承担“请求/应答型”职责；高频或异步流转交给 WebSocket。

适合继续保留在 HTTP 的内容：

- bootstrap
- health
- property/resource write
- command invoke
- 配置、审计、管理类接口

适合迁移到 WebSocket 的内容：

- runtime state delta
- alarms / events
- command result lifecycle
- connection and adapter health push



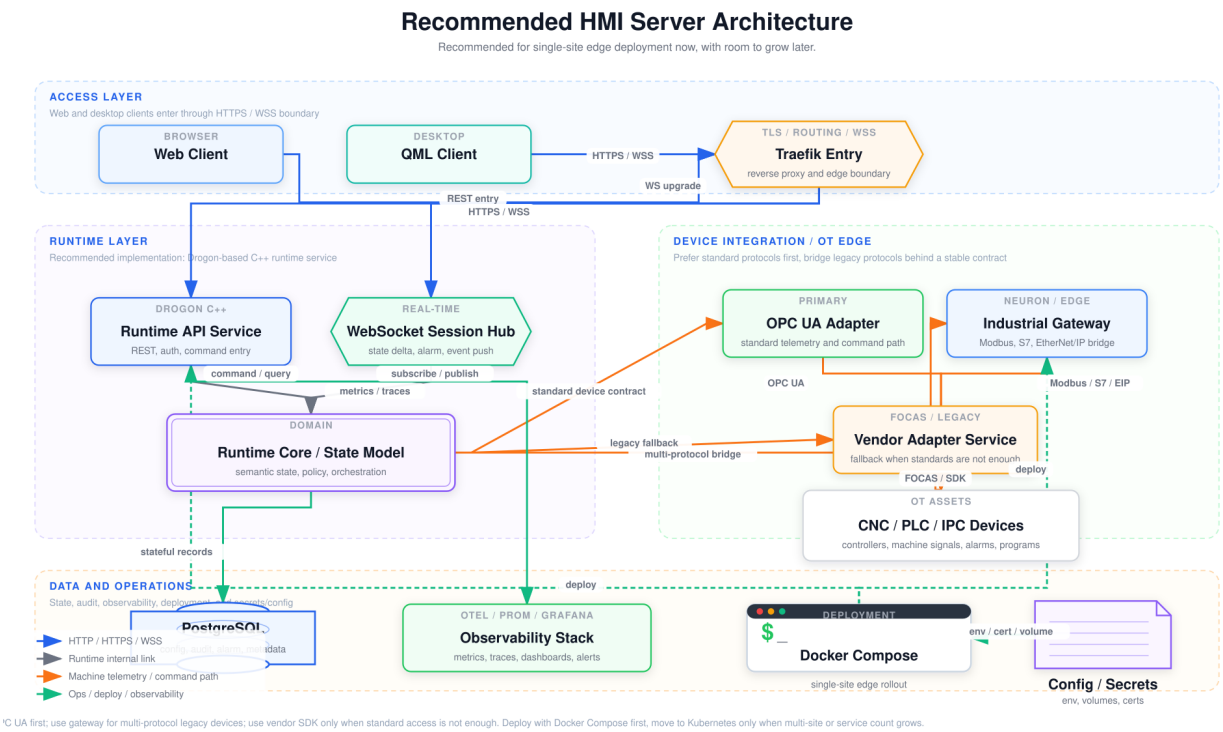
图文件：

- PNG
- SVG

1.4 4. 推荐目标架构

推荐的整体形态如下：

1. Web Client 和 QML Client 统一通过 Traefik 进入 HTTPS / WSS 边界。
2. Runtime API Service 负责 REST、鉴权、命令入口和管理接口。
3. WebSocket Session Hub 负责实时状态、告警和事件推送。
4. Runtime Core / State Model 负责语义状态、策略、权限边界和指令编排。
5. southbound 侧优先通过 OPC UA Adapter 对接标准化设备。
6. legacy 协议通过 Industrial Gateway 收口，例如 Modbus / S7 / EtherNet/IP。
7. 厂商标准不够时，再增加单独的 Vendor Adapter Service，例如 FOCAS / SDK。
8. PostgreSQL 存配置、审计、报警、程序元数据和运营数据。
9. OpenTelemetry + Prometheus + Grafana 负责观测。
10. 单站点先用 Docker Compose，多站点和服务规模扩大后再考虑 Kubernetes。



图文件：

- PNG
- SVG

1.5 5. 方案对比

方案	Northbound	Southbound	部署/管理
A. C++ 一体化边缘服务	Drogon + HTTP/WS	open62541 + libmodbus + libplctag + vendor SDK	Compose +
B. 工业网关 + HMI Runtime	Drogon/oat++ + HTTP/WS	设备先接工业网关，再由 runtime 消费统一数据	Compose +
C. 平台化微服务	前端 HTTP/WS，服务间 gRPC	adapter/gateway/driver 服务化	Kubernete

1.6 6. 最终推荐

建议采用 A + B 的混合方案：

- northbound 保持自研 runtime，但 transport 换成成熟框架
- southbound 优先标准协议，legacy 协议尽量经工业网关收口
- 厂商 SDK 只作为补充，不作为第一选择

推荐技术栈如下：

1.6.1 6.1 Runtime / Northbound

- C++20
- Drogon
- HTTPS / REST / WebSocket
- Traefik

1.6.2 6.2 Device Integration / Southbound

- OPC UA 优先
- open62541 作为自研适配基础
- 工业网关收口 Modbus / Siemens S7 / EtherNet/IP
- 厂商 SDK / 专有协议作为 fallback

1.6.3 6.3 Data / Ops

- PostgreSQL
- OpenTelemetry
- Prometheus
- Grafana
- Docker Compose

1.7 7. 协议与集成策略

推荐接入优先级:

1. 优先判断设备是否提供 OPC UA Server
2. 如果是机床监控和生产数据统一，可评估 MTConnect
3. 如果现场协议种类多、品牌杂，优先上工业网关
4. 只有在标准链路覆盖不了控制或语义时，再增加专有 adapter

不建议的做法:

- 不要把 PLC/CNC 原始地址直接暴露给前端
- 不要把所有协议 driver 直接堆进一个 monolith 进程
- 不要把高频状态同步长期建立在 polling 上
- 不要让 MQTT / WS 直接承担安全关键运动控制

1.8 8. 实施清单

1.8.1 第一阶段：必须做

- 把当前 hand-written HTTP transport 替换为 Drogon
- 明确 northbound contract：REST 和 WS 的边界
- 建立 Runtime Core / State Model 与 southbound adapter 的分层
- 接入 Traefik，完成 HTTPS / WSS 入口
- 选定 PostgreSQL 作为主库
- 打通最小观测链路：OpenTelemetry + Prometheus + Grafana
- 用 Docker Compose 把 runtime、db、proxy、observability 跑通

1.8.2 第二阶段：建议尽快做

- 做 OPC UA Adapter
- 如果现场设备品牌复杂，引入工业网关收口 legacy 协议
- 给 runtime 增加 adapter 健康、连接状态、告警推送
- 给命令执行增加审计和异步结果生命周期
- 给部署加 env / volume / cert 规范

1.8.3 第三阶段：按规模再做

- 时序数据量上来后，再评估专门时序库
- 出现多站点或多服务规模后，再考虑 Kubernetes
- 设备协议增长到维护困难时，再继续拆 adapter 服务

1.9 9. 本次整理产物

- 方案文档：hmi-server-recommendation.md
- PDF 导出版：hmi-server-recommendation.pdf
- 通信模式图：assets/hmi-server-recommendation/hmi-server-transport-current-vs-target.{png,svg}
- 推荐架构图：assets/hmi-server-recommendation/hmi-recommended-target-architecture.{png,svg}

1.10 10. 参考

- Drogon: https://drogon.org/
- open62541 / OPC UA: https://open62541.org/doc/master/
- libmodbus: https://libmodbus.org/
- libplctag: https://libplctag.github.io/
- MTConnect: https://www.mtconnect.org/
- EMQX Neuron: https://www.emqx.com/en/products/emqx-neuron
- Traefik: https://doc.traefik.io/traefik/master/expose/overview/
- Docker Compose: https://docs.docker.com/guides/docker-compose/
- Kubernetes: https://kubernetes.io/docs/concepts/overview/
- OpenTelemetry C++: https://opentelemetry.io/docs/languages/cpp/
- Prometheus: https://prometheus.io/docs/tutorials/getting_started/
- Grafana Docker: https://grafana.com/docs/grafana/latest/setup-grafana/installation/docker/
- gRPC C++: https://grpc.io/docs/languages/cpp/
- PostgreSQL: https://www.postgresql.org/docs/current/index.html